OntarioTech Engineering & Applied Science

IEEE ISORC 2023
May 23-25

A Robust Scheduling Algorithm for Overload-Tolerant Real-Time Systems

Amin Avan, Akramul Azim, Qusay Mahmoud

Introduction:

- Tasks in a real-time system has ‘deadline’ & ‘response time’.
  - The consequences of missing deadlines are tolerable in a soft real-time system.
    - e.g., multimedia devices, monitoring equipment, interactive games, virtual reality.
- Workload utilization (U):
  - If ($U \leq 1$) → workload is feasible.
  - If ($U > 1$) → workload is infeasible.
- The system is ‘overloaded’ when ($U > 1$).
- Robust algorithm can handle overloading for satisfactory results.

Related Work:

Algorithm	Features
Rate-Monotonic (RM)	• Static scheduling algorithm [1]. • Select task with shorter period. • Simple implementation [2].
Earliest Deadline First (EDF)	• Dynamic scheduling algorithm [3]. • Select task with earliest deadline [3]. • performs optimally with (U<=1) [1].
Least Laxity First (LLF)	• Dynamic scheduling algorithm [3]. • Select task with lowest laxity [3]. • performs optimally with (U<=1) [1].
Modified Least Laxity First (MLLF)	• Schedule like LLF, but switch to EDF when 'laxity-tie' occurs.

Related Work (cont.):

Algorithm	Features
Joint-EDF-RM (JER)	• Schedule based on EDF in “non-overloading” condition [6]. • Schedule based on RM in “overloading” condition [6].
Robust Earliest Deadline (RED)	• Schedule like EDF in non-overloading condition [6]. • Rejects a task with sufficient utilization to decrease workload utilization to non-overloaded.
Constant Bandwidth Server (CBS)	• Prevent sporadic tasks from exceeding WCET to avoid overloading [8]. • Utilized in the Linux kernel [9].

Problem statement:

- Workload utilization (U):
  - U is sum of all task utilization in a workload.
  - The workload can be executed in a processor if (U<1).
- A soft real-time system may experience an “overloading (U>1)” condition.
  - (U>1) means the processing resource of the system cannot meet the processing demands of all the tasks in the workload.
- Real-time systems such as:
  - Multimedia devices, monitoring equipment, interactive games, virtual reality may encounter overloading situations.
    - (U>1) is due to:
      - Cascades of system failures
      - Emerging overwhelming numbers of tasks
      - Sudden changes in the environment
      - Flawed system design

Weakness of Related Work:

Algorithm	Limitation on handling the overload situation
RM	• Tasks miss their deadlines when ($0.69<U\leq1$), let alone ($U>1$). [10].
EDF	• “Domino Effect” occur → all tasks miss their deadlines [2].
LLF	• “Laxity-tie” problem → ‘context switching’ overhead [12].
RED	• Instant tasks rejection [13]. • Reject tasks with high utilization instead of rejecting multiple low-utilization tasks [13].

Proposed Scheduling Algorithm (EPD):

- EPD handles overloads with minimal deadline misses without ‘context switching’ & ‘domino effect’.
- EPD calculates execution possibility (EPτ_i) of the task with earliest deadline at each time unit.

$$EP_{\tau_i} = \frac{\text{remaining execution time of } \tau_i}{\text{remaining time to deadline of } \tau_i}$$

- EPD selects τ_i if (EPτ_i ≤1), indicating the task can be finished before its deadline.

Example:

For (t=1) EDF select τ_1

For (t=1) EPD select τ_1 as EPτ_1= (2/4) ≤ 1

Workload "X"

τ_i	A_i	C_i	$(P_i = D_i)$
τ_1	0	2	4
τ_2	0	3	6
τ_3	0	2	8

τ_1	τ_1	τ_1				τ_1	τ_1			τ_1	τ_1				τ_1	τ_1						τ_1	τ_1	
τ_2			τ_2	τ_2	τ_2							τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			
τ_3								τ_3	τ_3								τ_3	τ_3						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EDF.

τ_1	τ_1	τ_1				τ_1	τ_1				τ_1	τ_1	τ_1	τ_1				τ_1	τ_1					
τ_2			τ_2	τ_2	τ_2			τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			τ_2	τ_2	τ_2		
τ_3																								
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EPD.

Example:

For (t=3) EDF select τ_2

For (t=3) EPD select τ_2 as EPτ_2= (3/4) ≤ 1

Workload "X"

τ_i	A_i	C_i	$(P_i = D_i)$
τ_1	0	2	4
τ_2	0	3	6
τ_3	0	2	8

τ_1	τ_1	τ_1				τ_1	τ_1			τ_1	τ_1				τ_1	τ_1						τ_1	τ_1	
τ_2			τ_2	τ_2	τ_2							τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			
τ_3								τ_3	τ_3								τ_3	τ_3						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EDF.

τ_1	τ_1	τ_1				τ_1	τ_1				τ_1	τ_1	τ_1	τ_1				τ_1	τ_1					
τ_2			τ_2	τ_2	τ_2			τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			τ_2	τ_2	τ_2		
τ_3																								
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EPD.

Example:

For (t=6) EDF select τ_1

For (t=6) EPD select τ_1 as EPτ_1= (2/3) ≤ 1

Workload "X"

τ_i	A_i	C_i	$(P_i = D_i)$
τ_1	0	2	4
τ_2	0	3	6
τ_3	0	2	8

τ_1	τ_1	τ_1				τ_1	τ_1			τ_1	τ_1				τ_1	τ_1						τ_1	τ_1	
τ_2			τ_2	τ_2	τ_2							τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			
τ_3								τ_3	τ_3								τ_3	τ_3						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EDF.

τ_1	τ_1	τ_1				τ_1	τ_1				τ_1	τ_1	τ_1	τ_1				τ_1	τ_1					
τ_2			τ_2	τ_2	τ_2			τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			τ_2	τ_2	τ_2		
τ_3																								
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EPD.

Example:

For (t=8) EDF select τ_3

For (t=8) EPD does not select τ_3 as EPτ_3= (2/1) !≤ 1; thus, EPD select the task with second earliest deadline that is τ_2 as EPτ_2= (3/5) !≤ 1

Workload "X"

τ_i	A_i	C_i	$(P_i = D_i)$
τ_1	0	2	4
τ_2	0	3	6
τ_3	0	2	8

τ_1	τ_1	τ_1				τ_1	τ_1			τ_1	τ_1				τ_1	τ_1						τ_1	τ_1	
τ_2			τ_2	τ_2	τ_2							τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			
τ_3								τ_3	τ_3								τ_3	τ_3						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

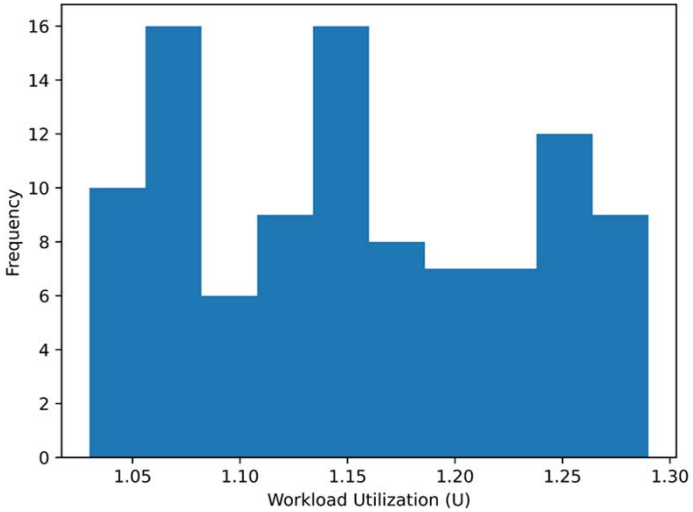
Task scheduling based on EDF.

τ_1	τ_1	τ_1				τ_1	τ_1				τ_1	τ_1	τ_1	τ_1				τ_1	τ_1					
τ_2			τ_2	τ_2	τ_2			τ_2	τ_2	τ_2					τ_2	τ_2	τ_2			τ_2	τ_2	τ_2		
τ_3																								
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

Task scheduling based on EPD.

Experimental Results:

- We implement algorithms in Python & evaluate their performance using 100 different overloaded workloads.
- The dataset contains workloads with different numbers of tasks.
  - Tasks in workloads: 2, …, 30
    - Deadlines: 4, …, 1000
    - Execution time: 1, …, 150
  - U: 1.03, …, 1.29
- Performance evaluation metrics include:
  - Task Miss Rate (MR)
  - Context Switching Rate (CSR)
  - Scheduling Algorithm Efficiency (SAE)
  - Effective Processor Utilization (EPU)



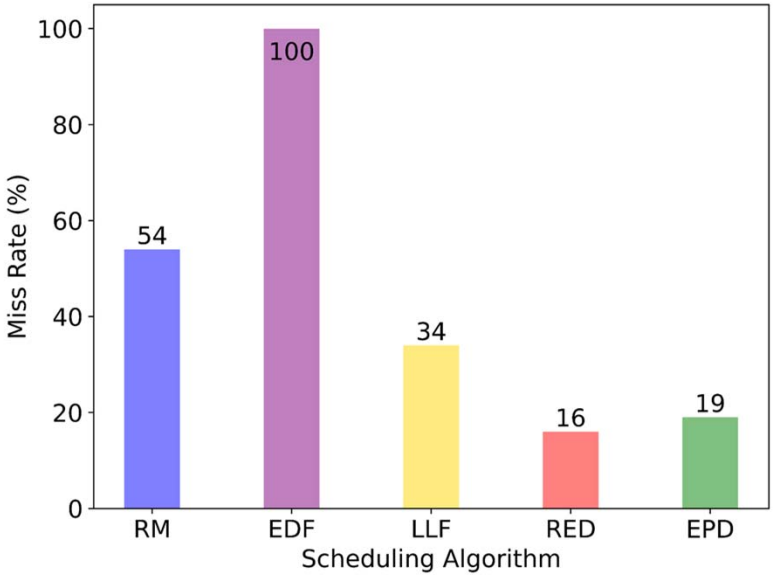
Distribution of workload utilization in the dataset.

Experimental Results – Task Miss Rate (MR):

- MR shows the percentage of missed deadline tasks in the workload.

$$MR = \frac{\text{number of task that miss their deadlines}}{\text{total number of task in the workload}}$$

- The MR of EPD is 44%, 64%, and 81% lower than LLF, RM, and EDF.
- Both EPD & RED achieve approximately the same result in MR.



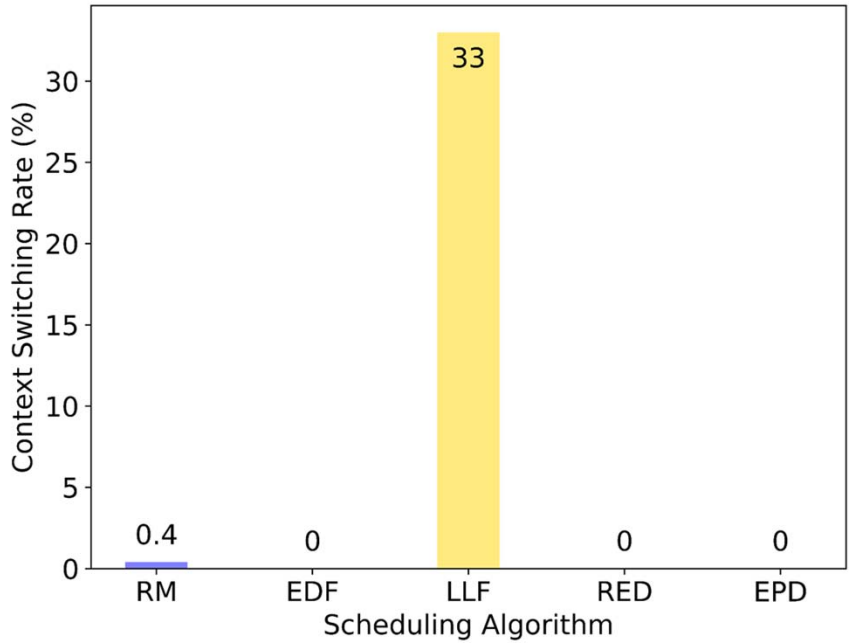
Task Miss Rate of Scheduling Algorithms.

Experimental Results – Context Switching Rate (CSR):

- ‘Context switching’ is performance cost of switching between unfinished tasks.

$$CSR = \frac{\text{number of context switching}}{\text{hyperperiod}}$$

- LLF suffers from context switching due to laxity-tie.
- EDF, RED, and EPD algorithms are resistant to context switching.



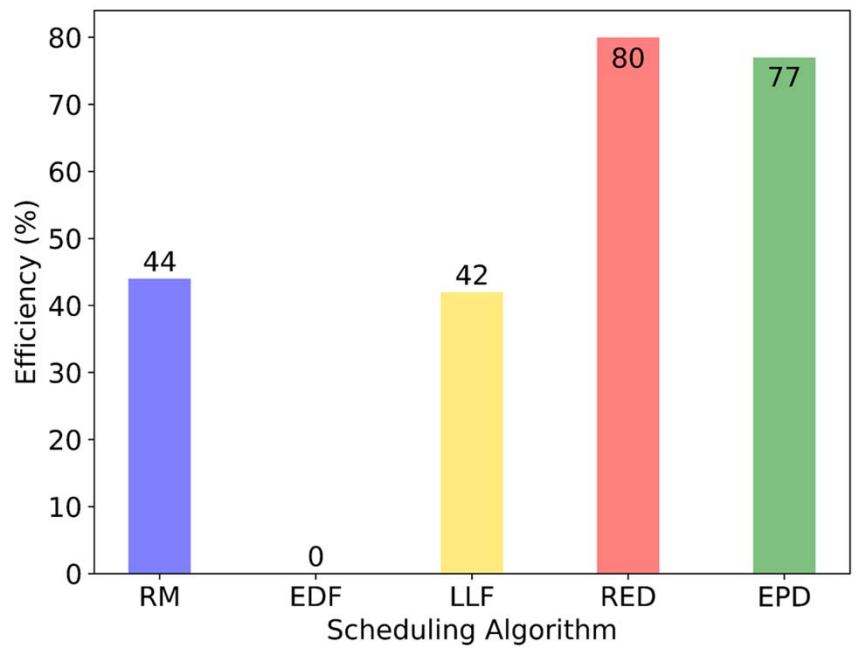
Context Switching Rate of Scheduling Algorithms.

Expt. Results – Scheduling Algorithm Efficiency (SAE):

- SAE considers both ‘context switching’ & ‘hit ratio’.

$$SAE = (\frac{N_c}{N_T}) \times (1 - \frac{N_{cs}}{H})$$

- Tasks miss deadlines with EDF.
- RM achieves 50% MR but outperforms LLF in SAE due to less context switching.
- High context switching in LLF decreases SAE.
- EPD achieves 42% & 45% higher SAE than RM & LLF.
- Both RED & EPD exhibit remarkable SAE.



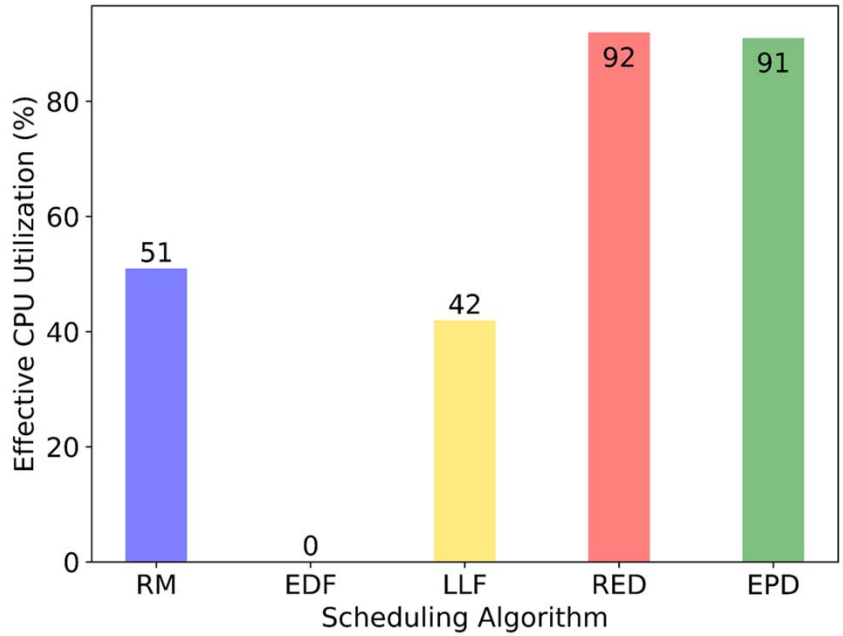
SAE of Scheduling Algorithms.

Expt. Results – Effective Processor Utilization (EPU):

- EPU assesses algorithm efficiency in utilizing processor capacity [14].

$$EPU = (\sum_{i=0}^{m} u_i) - (\frac{N_{cs}}{H})$$

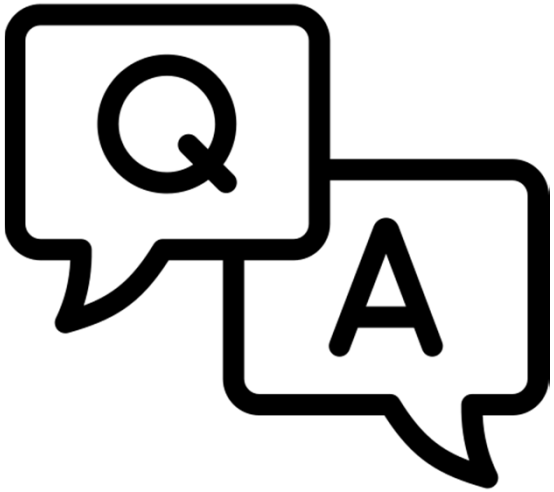
- EPU: total utilization of 'm' finished tasks subtracted by context-switching.
- EPD has the highest EPU among RM, EDF, LLF.
- RED and EPD achieve similar results.



EPU of Scheduling Algorithms.

Conclusion & Future Work:

- Real-time systems may experience overloading situations; thus, we propose EPD to handle overloads.
- RM, EDF, and LLF are classic algorithms that suffer from:
  - Processor utilization limitation
  - Domino effect
  - Context switching
- EPD addresses the three mentioned issues by executing tasks with the earliest deadline if (EP ≤ 1).
- EPD outperforms RM, EDF, LLF, and provides similar results compared to RED.
- Implementing EPD on multiprocessor systems.
- Implementing EPD for task scheduling of Edge users in edge computing.



Thanks for your time and attention.

Amin Avan: amin.avan@ontariotechu.net

Akramul Azim: akramul.azim@ontariotechu.ca

Qusay Mahmoud: Qusay.Mahmoud@ontariotechu.ca

References:

1. Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. Journal of the ACM (JACM), 20(1), 46-61.
2. Sharma, R., Nitin, N., AlShehri, M. A. R., & Dahiya, D. (2021). Priority-based joint EDF–RM scheduling algorithm for individual real-time task on distributed systems. The Journal of Supercomputing, 77, 890-908.
3. Dertouzos, M. L. (1974). Control robotics: The procedural control of physical processes. In Proceedings IF IP Congress, 1974.
4. Mok, A. K. L. (1983). Fundamental design problems of distributed systems for the hard-real-time environment (Doctoral dissertation, Massachusetts Institute of Technology).
5. Oh, S. H., & Yang, S. M. (1998, October). A modified least-laxity-first scheduling algorithm for real-time tasks. In Proceedings Fifth International Conference on Real-Time Computing Systems and Applications (Cat. No. 98EX236) (pp. 31-36). IEEE.
6. Sharma, R. (2014). Performance evaluation of new joint EDF-RM scheduling algorithm for real time distributed system. Journal of Engineering, 2014.
7. Buttazzo, G. C., & Stankovic, J. A. (1993). RED: Robust earliest deadline scheduling (pp. 100-111). University of Massachusetts, Department of Computer Science.
8. Abeni, L., & Buttazzo, G. (1998, December). Integrating multimedia applications in hard real-time systems. In Proceedings 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279) (pp. 4-13). IEEE.
9. Abeni, L., Lipari, G., & Lelli, J. (2015). Constant bandwidth server revisited. Acm Sigbed Review, 11(4), 19-24.
10. Zhou, J. (2020). Real-time task scheduling and network device security for complex embedded systems based on deep learning networks. Microprocessors and Microsystems, 79, 103282.
11. Dertouzos, M. L., & Mok, A. K. (1989). Multiprocessor online scheduling of hard-real-time tasks. IEEE Transactions on software engineering, 15(12), 1497-1506.
12. Suman, C., & Kumar, G. (2019, March). Performance Enhancement of Real Time System using Dynamic Scheduling Algorithms. In 2019 IEEE 5th International Conference for Convergence in Technology (I2CT) (pp. 1-6). IEEE.
13. Buttazzo, G. C. (2011). Hard real-time computing systems: predictable scheduling algorithms and applications (Vol. 24). Springer Science & Business Media.
14. Teraiya, J., & Shah, A. (2020). Analysis of dynamic and static scheduling algorithms in soft real-time system with its implementation. In Soft Computing: Theories and Applications: Proceedings of SoCTA 2018 (pp. 757-768). Springer Singapore.